TP4 : Introduction à l'algorithmique et à Python

NOM ETUDIANT :

Types structurés, Sous-programmes et Programme principal

Chaque notebook de TP est à déposer sur moodle dans la zone de dépôt dédiée, au plus tard, la semaine suivant le TP ou selon la date spécifiée sur moodle. Ce dépôt tient lieu de compte-rendu et sera évalué.

Le nom de votre fichier .ipynb doit contenir votre nom. Exemple : TP1_PYTHON_DUPONT.ipynb

Bonnes pratiques :

- les noms de variables et de constantes doivent être explicites sauf s'il existe des conventions par exemple i,j,k pour les indices ;
- le code doit être commenté lorsque les actions réalisées sont complexes les commentaires doivent refléter l'algorithme sous-jacent ;
- chaque programme doit faire l'objet des tests des cas limite et d'un rapport de tests tester le fonctionnement normal, ainsi que les cas d'erreur potentiels.

Le code est écrit dans une cellule code et les rapports de tests par copie-coller des résultats des tests successifs, et ce, en dessous du texte de l'exercice.

Un rapport de test s'accompagne de message explicitant la demande de saisie ou bien la nature de la valeur affichée. On doit comprendre la nature du test effectué.

EXERCICE 5 : Programme python

Rassembler le code des différentes fonctions écrites et testées dans le TP3

Entrée [ ]:

```
# Biblothèques nécessaires au TP

# à compléter
```

Entrée [ ]:

```
#################################################################
# SOUS-PROGRAMMES DE GESTION DES OUVERTURES ######################
#                                           ######################
# ajouter_ouverture, supprimer_ouvertures   ######################
#################################################################

# à compléter
```

Entrée [ ]:

```
#################################################################
# SOUS-PROGRAMMES DE GESTION DES OBSTACLES  ######################
#                                           ######################
# ajouter_obstacle, supprimer_obstacles     ######################
#################################################################

# à compléter
```

Entrée [ ]:

```
#########################################################################################
# SOUS-PROGRAMMES DE TRACAGE DES ELEMENTS DE L'ENVIRONNEMENT  ######################
#                                                             ######################
# tracer_piece, tracer_ouvertures_piece                       ######################
# tracer_obstacles_piece, tracer_environnement                ######################
#########################################################################################

# à compléter
```

Entrée [ ]:

```
##########################################################################################
# INITIALISATION ET AFFICHAGE DE L'ENVIRONNEMENT                    ####################
#                                                                   ####################
# initialiser_environnement, afficher_info_environnement            ####################
##########################################################################################
# ces deux sous-programmes seront à utiliser à la place de la création faite dans  #
# le TP3 -- Cela peut simuler la lecture des informations dans un fichier          #
##########################################################################################

def initialiser_environnement() :

    piece = { 'nom' : 'entrepot' , 'dimensions' : (400,400), 'coin_HD' : (200,200),
          'couleur' : 'blue', 'trait' : 5, 'ouvertures': [], 'obstacles' : []}

    ouverture = {'nom': 'porte1', 'largeur': 60, 'distance_porte_angle' :  100, 'coi
              'pos_porte' : (), 'couleur' : 'orange', 'trait' : 10 }

    obstacle = {'nom': 'obstacle1', 'type': 'rond', 'centre': (50,50), 'dim' : 20, '


    ajouter_ouverture (ouverture, piece)

    ajouter_obstacle (obstacle, piece)

    return piece

########

def afficher_info_environnement(ref_env):

    print('PIECE ------------------------' )
    for key in ref_env.keys() :
        if key not in ['ouvertures', 'obstacles']:
            print('\t', key, ' = ', ref_env[key])

    nb_ouvertures = len(ref_env['ouvertures'])
    nb_obstacles = len(ref_env['ouvertures'])
    for ouv in range (nb_ouvertures):
        print('OUVERTURE ', ouv+1, ' = ', ref_env['ouvertures'][ouv])
    for obs in range(nb_obstacles):
        print('OBSTACLE ', obs+1, ' = ', ref_env['obstacles'][obs])

    return None

########
```

Entrée [ ]:

```
##########################################################################################
# SOUS-PROGRAMMES DE GESTION DES DEPLACEMENT DU ROBOT.              ####################
#                                                                   ####################
# ajouter_trajectoire, supprimer_trajectoire                        ####################
##########################################################################################

# à compléter
```

Entrée [ ]:

```
##########################################################################################
# SOUS-PROGRAMMES DE GESTION DE LA MISSION DU ROBOT                    ###################
#                                                                      ###################
# definir_mission_robot donnée ci-après pour définir le robot          ###################
# et la trajectoire à effectuer lors de la mission                     ###################
#                                                                      ###################
# valider_position pour lisser les petites différences < delta         ###################
# entre la position courante et la positions attendue                  ###################
##########################################################################################

def definir_mission_robot(ref_env):

    # Caractéristiques du robot à utiliser
    robot = { 'nom' : 'r2d2', 'couleur':'red', 'trait' : 3, 'pos_robot': (), 'sens':

    # Définition de la trajectoire
    trajectoire = [ [(250,90), 2, 100, ()], [(150,90), 1, 50, ()], [(150, 140), 2, 2
                [(-100, 140), 3, 100, ()], [ (-100, 40), 0, 100, ()] ]

    ajouter_trajectoire(robot, trajectoire)

    nb_ouvertures = len(ref_env['ouvertures'])
    if nb_ouvertures != 0 :
        ouverture_entree = ref_env['ouvertures'][0]
        X_robot_depart = ouverture_entree['pos_porte'][0]
        Y_robot_depart = ouverture_entree['pos_porte'][1]
        X_robot_depart = X_robot_depart + 50
        Y_robot_depart = Y_robot_depart + ouverture_entree['largeur']/2

        robot['pos_robot'] = (X_robot_depart, Y_robot_depart)

        robot['sens'] = 2
    return robot

# à compléter
```

Entrée [ ]:

```
def valider_position(ref_position_courante, ref_debut_segment, delta):
    validite_OK = False

    x1,y1 = ref_position_courante
    print('TRACE x1 y1 = ', x1, y1)
    x2,y2 = ref_debut_segment
    print('TRACE x2, y2 = ', x2, y2)
    intervaleX2 = range(x2-delta, x2+delta)
    intervaleY2 = range(y2-delta, y2+delta)
    if x1 in intervaleX2 and y1 in intervaleY2 :
        validite_OK = True
    return validite_OK
```

Entrée [ ]:

```
##############################################################################
# à compléter avec                                                  ##########
# modifier_orientation, positionner_robot, exécuter_trajectoire ##########
```

Entrée [ ]:

```
#########################################################################
# PROGRAMME PRNCIPAL A EXECUTER                          ###############
#########################################################################

mode_trace = True              # permet d'activer ou désactiver le mode trace
                               # modifier vos codes en conséquence pour que
                               # les messages de trace s'affichent ou pas suivant le

environnement = initialiser_environnement()

if mode_trace :
        print('TRACE =', environnement)  # S'affiche uniquement si le mode trace est

afficher_info_environnement(environnement)

tracer_environnement(environnement)

robot = definir_mission_robot(environnement)

executer_trajectoire(robot, environnement)
```

```
Une fois validé, rassembler l'ensemble du code commenté dans un fichier
NOM_ETUD_robot.py de façon à pouvoir l'exécuter directement depuis le shell avec la
commande

python3 NOM_ETUD_robot.py

Tester votre programme. il doit fonctinner exactement comme le code de ce notebook.
Déposer le notebook, le fichier .py, la version pdf du notebook (avec les traces
des exécutions) et la capture écran du résultat obtenu
```